

Laboratoire 1

Bases Linux, programmes Set-UID et variables d'environnement

Amel Moumene 300362442

Matthew Roger 3003368930

Ibrahima Badiane 300380670

CEG 4799 A00 / CSI 4539 A00 — 2 octobre 2026

1. Introduction et modèle de menace

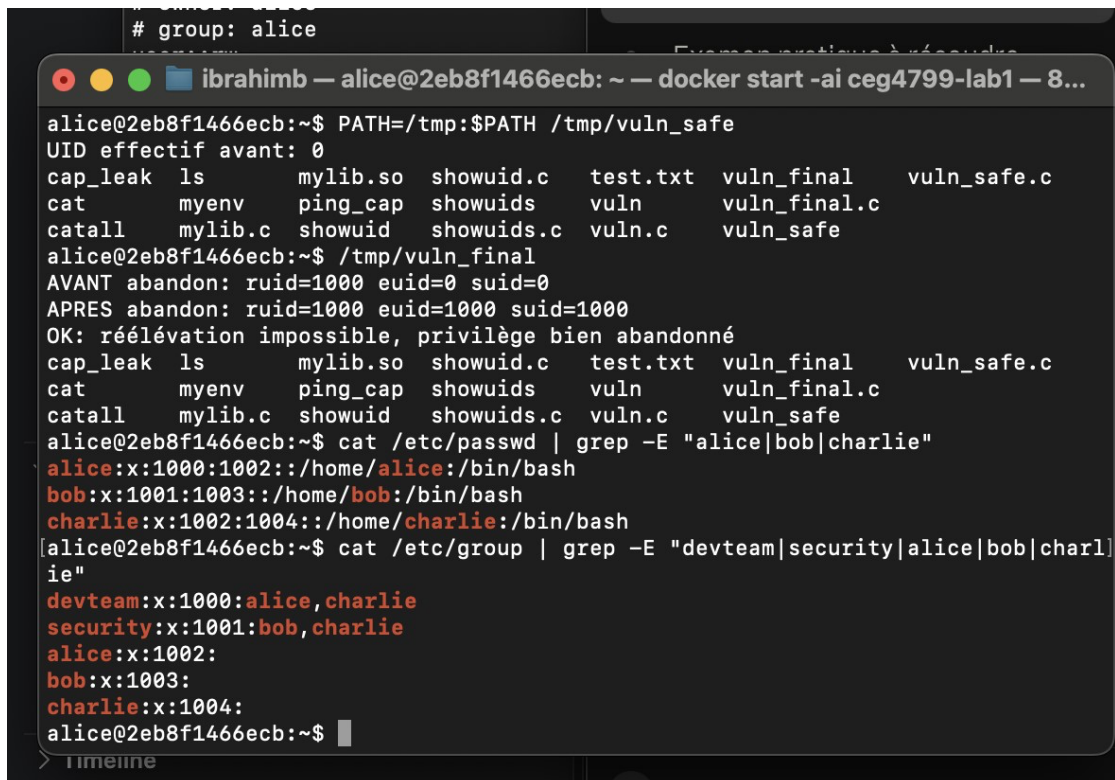
L'objectif est de configurer le contrôle d'accès Linux, d'analyser les entrées d'un programme privilégié, puis de sécuriser ses invocations externes et d'abandonner les privilèges inutiles. Les actifs à protéger sont les fichiers, leur intégrité et les droits associés à root dans l'environnement du laboratoire.

Le modèle de menace considère un utilisateur local capable de fournir des arguments, de modifier son environnement et de créer de faux exécutables dans /tmp. Les captures proviennent du conteneur ceg4799-lab1, sous le compte alice. Les essais concernent catall, vuln, vuln_safe et vuln_final. Les résultats décrits ci-dessous sont ceux des captures E1 à E8 ; les anciens résultats du dossier resultats/ utilisent d'autres comptes et ne sont pas fusionnés avec cette série.

2. Contrôle d'accès — E1 : utilisateurs et groupes

La lecture de /etc/passwd et de /etc/group confirme trois utilisateurs. Alice (UID 1000, GID 1002) appartient à devteam ; Bob (UID 1001, GID 1003) à security ; Charlie (UID 1002, GID 1004) aux deux groupes. Devteam a le GID 1000 et security le GID 1001. Chaque compte dispose de son répertoire personnel et du shell /bin/bash.

Le champ x de /etc/passwd renvoie au stockage protégé des informations de mot de passe dans /etc/shadow. Le sel rend différentes les empreintes de mots de passe identiques et limite les attaques par tables précalculées ; il n'a pas besoin d'être secret. Les groupes permettent d'attribuer les accès par rôle sans donner les droits de root.



```
# group: alice
ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
alice@2eb8f1466ecb:~$ PATH=/tmp:$PATH /tmp/vuln_safe
UID effectif avant: 0
cap_leak ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv   ping_cap  showuids  vuln       vuln_final.c
catall   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$ /tmp/vuln_final
AVANT abandon: ruid=1000 euid=0 suid=0
APRES abandon: ruid=1000 euid=1000 suid=1000
OK: réélévation impossible, privilège bien abandonné
cap_leak ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv   ping_cap  showuids  vuln       vuln_final.c
catall   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$ cat /etc/passwd | grep -E "alice|bob|charlie"
alice:x:1000:1002::/home/alice:/bin/bash
bob:x:1001:1003::/home/bob:/bin/bash
charlie:x:1002:1004::/home/charlie:/bin/bash
alice@2eb8f1466ecb:~$ cat /etc/group | grep -E "devteam|security|alice|bob|charlie"
devteam:x:1000:alice,charlie
security:x:1001:bob,charlie
alice:x:1002:
bob:x:1003:
charlie:x:1004:
alice@2eb8f1466ecb:~$
```

Figure 1 — Comptes alice, bob et charlie et appartenances aux groupes.

2. Contrôle d'accès — E2 et E3

E2 — Permissions, umask et sticky bit

La commande `umask` affiche 0002. Pour une création sans ACL par défaut, le mode demandé est filtré par le complément du masque : un fichier demandé en 0666 devient 0664 et un dossier demandé en 0777 devient 0775. Cette règle concerne la création ; elle ne modifie pas les fichiers déjà présents.

Le dossier `/shared` appartient à `alice:devteam` et présente le mode `drwxrwxr-t`, soit 1775. Alice et le groupe disposent de `rwx` ; les autres disposent de `r-x`. Le `t` indique le sticky bit : la suppression ou le renommage d'un fichier est réservé au propriétaire du fichier, au propriétaire du dossier ou à un processus disposant du privilège approprié. Il ne retire pas les droits d'écriture sur le contenu du fichier. Aucun bit Set-GID n'apparaît ici.

Cette configuration permet le travail du groupe tout en limitant la suppression des fichiers d'autrui. Elle laisse cependant les autres utilisateurs lister et traverser le dossier. La capture montre l'état final ; elle ne présente pas un essai de suppression ni la création comparative de fichiers. L'erreur `umaskls` est une faute de saisie, suivie des commandes correctes.

E3 — ACL et capacité ciblée

`getfacl /home/alice/secret.txt` affiche `user::rw-, user:bob:r--, group::---, mask::r--` et `other::---`. Bob reçoit donc une permission de lecture explicite sans rejoindre le groupe propriétaire `alice`. Le masque `r--` conserve ce droit et n'accorde aucune écriture. Les permissions de traversée des dossiers parents restent nécessaires pour accéder au fichier. [5]

`getcap -r /tmp/` affiche `/tmp/ping_cap = cap_net_raw+ep`. Une seule capacité de fichier est affichée : `CAP_NET_RAW`, prévue pour les sockets brutes et de type paquet. Son attribution cible un besoin précis, contrairement à un UID effectif `root` donnant des pouvoirs beaucoup plus larges. La capture atteste cette attribution ; le retrait du bit Set-UID et l'exécution fonctionnelle ne sont pas visibles dans cet extrait. [6]

```
ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
-rw-r----- 1 alice alice 17 Sep 29 18:41 secret.txt
alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
alice@2eb8f1466ecb:~$ ls -la /home/alice/
total 44
drwxr-xr-x 3 alice alice 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 28 22:48 ..
-rw-r----- 1 alice alice 4267 Sep 29 18:05 .bash_history
-rw-r--r-- 1 alice alice 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 alice alice 3771 Feb 25 2020 .bashrc
-rw-r--r-- 1 alice alice 807 Feb 25 2020 .profile
drwxr-xr-x 5 alice alice 4096 Sep 29 18:04 CEG4799-Lab1
-rwxrwxr-x 1 alice alice 96 Sep 29 15:56 exploit_e6a.sh
-rwxrwxr-x 1 alice alice 194 Sep 29 15:56 exploit_e6b.sh
-rw-r----- 1 alice alice 17 Sep 29 18:41 secret.txt
alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
alice@2eb8f1466ecb:~$ umask
0002
alice@2eb8f1466ecb:~$ ls -la /shared
total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 29 18:58 ..
alice@2eb8f1466ecb:~$
```

Figure 2 — Umask 0002 et dossier partagé avec sticky bit.

```
ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
-rwxrwxr-x 1 alice alice 194 Sep 29 15:56 exploit_e6b.sh
-rw-r----- 1 alice alice 17 Sep 29 18:41 secret.txt
alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
alice@2eb8f1466ecb:~$ umask
0002
alice@2eb8f1466ecb:~$ ls -la /shared
total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 29 18:58 ..
alice@2eb8f1466ecb:~$ getfacl /home/alice/secret.txt
getfacl: Removing leading '/' from absolute path names
# file: home/alice/secret.txt
# owner: alice
# group: alice
user::rw-
user:bob:r--
group::---
mask::r--
other::---
alice@2eb8f1466ecb:~$ getcap -r /tmp/ 2>/dev/null
/tmp/ping_cap = cap_net_raw+ep
alice@2eb8f1466ecb:~$
```

Figure 3 — ACL de Bob et capacité de `ping_cap`.

Les figures 2 et 3 sont également reproduites en grand format dans les annexes A et B.

3. Analyse de la vulnérabilité — E4 et E5

E4 — Modèle de privilège

`/tmp/showuids` affiche un UID réel de 1000, un UID effectif de 0 et un UID sauvegardé de 0. L'UID réel identifie Alice ; l'UID effectif détermine les permissions courantes ; l'UID sauvegardé permet certaines transitions ultérieures. Ces valeurs correspondent au comportement attendu d'un programme Set-UID `root`. Le propriétaire et le mode du fichier ne sont pas affichés sur cette capture.

Dans ce modèle, le noyau applique le Set-UID à l'exécution du binaire : l'UID effectif devient celui du propriétaire et sa valeur est copiée dans l'UID sauvegardé. L'UID réel reste celui du lanceur. L'élévation ne se produit donc pas à l'appel de `system()` ni au moment du `chmod` préparatoire. [2]

E5 — Trois surfaces et hypothèses implicites

Surface	Hypothèse implicite et analyse
Arguments / données	H1 : l'argument fourni reste un nom de fichier. La chaîne /tmp/test.txt; id fait pourtant exécuter id : une donnée est interprétée comme syntaxe shell.
PATH / IFS	H2 : l'environnement du lanceur est fiable. PATH peut sélectionner un faux exécutable lorsqu'un nom relatif est utilisé. IFS agit sur certaines expansions du shell ; sa seule modification ne garantit pas un détournement.
Liaison dynamique	H3 : les fonctions chargées sont toujours celles attendues. LD_PRELOAD peut substituer des fonctions lors d'une exécution ordinaire ; le chargeur impose des restrictions en mode sécurisé.

La tentative LD_PRELOAD=/tmp/mylib.so /tmp/catal /tmp/test.txt affiche uniquement TEST ALICE. Aucun effet du préchargement n'est visible. C'est compatible avec le filtrage des chemins contenant / en mode sécurisé, mais la capture ne fournit pas de contrôle positif montrant la bibliothèque active sur un binaire ordinaire. [3]

```
total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41
drwxr-xr-x 1 root root 4096 Sep 29 18:58
alice@2eb8f1466ecb:~$ getfacl /home/alice/secret.txt
getfacl: Removing leading '/' from absolute path names
# file: home/alice/secret.txt
# owner: alice
# group: alice
user::rw-
user:bob:r--
group::---
mask::r--
other::---

alice@2eb8f1466ecb:~$ getcap -r /tmp/ 2>/dev/null
/tmp/ping_cap = cap_net_raw+ep
alice@2eb8f1466ecb:~$ /tmp/showuid
Real UID: 1000
Effective UID: 0
alice@2eb8f1466ecb:~$ /tmp/showuids
Real UID: 1000
Effective UID: 0
Saved UID: 0
alice@2eb8f1466ecb:~$
```

Figure 4 — UID réel, effectif et sauvegardé.

```
alice@2eb8f1466ecb:~$ getcap -r /tmp/ 2>/dev/null
/tmp/ping_cap = cap_net_raw+ep
alice@2eb8f1466ecb:~$ /tmp/showuid
Real UID: 1000
Effective UID: 0
alice@2eb8f1466ecb:~$ /tmp/showuids
Real UID: 1000
Effective UID: 0
Saved UID: 0
alice@2eb8f1466ecb:~$ # Surface 1 - injection via argument
alice@2eb8f1466ecb:~$ /tmp/catal "/tmp/test.txt; id"
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 2 - variable PATH
alice@2eb8f1466ecb:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 3 - liaison dynamique
alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catal /tmp/test.txt
TEST ALICE
alice@2eb8f1466ecb:~$
```

Figure 5 — Arguments, PATH et essai LD_PRELOAD.

Les figures 4 et 5 sont reproduites en grand format dans les annexes C et D.

4. Exploitations — E6

E6a / T4 — Injection de commande

La commande /tmp/catal "/tmp/test.txt; id", également illustrée par ~/exploit_e6a.sh, produit TEST ALICE puis la sortie de id. Le point-virgule déclenche une commande supplémentaire. La cause première est l'interprétation d'une entrée non fiable par un shell : la frontière entre argument et commande n'est pas conservée. Propriété violée : une entrée utilisateur doit rester une donnée et ne jamais ajouter une commande. [4]

La sortie de la commande injectée indique uid=1000(alice), gid=1002(alice), avec le groupe devteam. Elle prouve l'injection, mais ne montre pas une commande injectée exécutée avec l'UID effectif root. La copie de référence cataloriginal.c disponible localement utilise déjà /bin/cat ; un chemin absolu ne suffit donc pas à supprimer l'injection si system() est conservé. Cette copie n'identifie pas à elle seule le source exact du binaire photographié.

E6b / T5 — Substitution par PATH

L'exécution de ~/exploit_e6b.sh affiche UID effectif: 0 puis le marqueur FAUX LS. Ce marqueur démontre la sélection d'un exécutable substitué. La cause est la recherche d'une commande par nom relatif dans un PATH influençable. Propriété violée : l'identité de l'exécutable invoqué doit être indépendante de l'environnement utilisateur.

Le faux ls affiche lui aussi uid=1000(alice). Il faut distinguer le programme parent, qui annonce un UID effectif de 0, de la commande enfant observée sous Alice. Une baisse de privilège par le shell est une explication possible, à confirmer avec le source et le shell utilisés. Les captures établissent les deux détournements ; elles ne permettent pas d'attribuer

root aux commandes enfants. Les scripts sont visibles par leur exécution, mais leur contenu n'a pas été fourni avec les captures.

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
Saved UID: 0
alice@2eb8f1466ecb:~$ # Surface 1 — injection via argument
alice@2eb8f1466ecb:~$ /tmp/catall "/tmp/test.txt; id"
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 2 — variable PATH
alice@2eb8f1466ecb:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 3 — liaison dynamique
[alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catall /tmp/test.txt ]
TEST ALICE
[alice@2eb8f1466ecb:~$ ~/exploit_e6a.sh ]
=== T4 : Injection de commande dans catall ===
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
[alice@2eb8f1466ecb:~$ ~/exploit_e6b.sh ]
=== T5 : Détournement par PATH sur vuln ===
UID effectif: 0
=== FAUX LS — execute par: uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam) ===
alice@2eb8f1466ecb:~$

```

Figure 6 — Exécution des scripts d'injection et de substitution de ls par PATH.

5. Correction et vérification — E7

Invocation sûre des commandes externes

L'essai de correction lance `PATH=/tmp:$PATH /tmp/vuln_safe`. Le résultat affiche UID effectif avant: 0, puis une liste de fichiers. Le marqueur FAUX LS observé avant correction ne réapparaît pas. Le répertoire /tmp peut donc rester prioritaire dans l'environnement du lanceur sans reproduire le détournement visible de cet essai.

La correction exigée remplace `system()` par `execve()` et désigne explicitement le programme, par exemple `/bin/ls` pour le scénario montré. Les arguments sont transmis dans un tableau terminé par `NULL`, sans construction d'une commande destinée au shell. L'environnement doit être reconstruit par liste blanche, par exemple `PATH=/usr/bin:/bin`, IFS avec espace, tabulation et saut de ligne, et `LANG=C`. Aucune variable `LD_PRELOAD` du lanceur n'a à être recopiée. [2]

Pour la variante de lecture `catall`, la même approche consiste à invoquer `/bin/cat` avec un argument `--` puis le nom de fichier. Un point-virgule dans ce nom reste alors un caractère de l'argument. Cette explication décrit le mécanisme de correction attendu ; la capture seule ne révèle pas le contenu de `vuln_safe.c`, le tableau d'environnement ni l'appel exact utilisé.

Comparaison avant / après

Version	Observation
vuln, avant correction	Le script E6b provoque le marqueur FAUX LS ; le parent annonce EUID 0.
vuln_safe, après correction	Avec /tmp en tête de PATH, un listage apparaît sans le marqueur FAUX LS. EUID 0 est encore annoncé avant l'invocation.

Le test valide l'absence de substitution visible pour ce PATH. Le rejeu de la chaîne exacte de T4 sur la version corrigée de catall n'apparaît pas dans E7. La suppression de l'injection et l'assainissement complet de l'environnement doivent être reliés au source et à ce rejeu pour une preuve exhaustive.

```

alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 2 — variable PATH
alice@2eb8f1466ecb:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 3 — liaison dynamique
[alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catall /tmp/test.txt ]
TEST ALICE
[alice@2eb8f1466ecb:~$ ~/exploit_e6a.sh ]
=== T4 : Injection de commande dans catall ===
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
[alice@2eb8f1466ecb:~$ ~/exploit_e6b.sh ]
=== T5 : Détournement par PATH sur vuln ===
UID effectif: 0
=== FAUX LS — execute par: uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam) ===
[alice@2eb8f1466ecb:~$ PATH=/tmp:$PATH /tmp/vuln_safe ]
UID effectif avant: 0
cap_leak  ls      mylib.so  showuid.c  test.txt   vuln_final  vuln_safe.c
cat      myenv  ping_cap  showuids   vuln       vuln_final.c
catall   mylib.c showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$

```

Figure 7 — Test de vuln_safe avec PATH=/tmp:\$PATH ; absence du marqueur FAUX LS.

5. Correction et vérification — E8

Abandon définitif des privilèges

L'exécution de /tmp/vuln_final présente successivement les valeurs suivantes :

Moment	UID réel	UID effectif	UID sauvegardé
Avant abandon	1000	0	0
Après abandon	1000	1000	1000

Le programme affiche ensuite « OK: réélévation impossible, privilège bien abandonné », puis le listage. La trace montre que les trois identités deviennent celles d'Alice avant la sortie de la commande externe. Elle rapporte également le succès du contrôle de non-réélévation ; le code de ce contrôle et son errno ne sont pas affichés.

Le mécanisme demandé est setresuid(uid, uid, uid), où uid est l'identité réelle du lanceur. Mettre également l'UID sauvegardé à cette valeur évite de conserver un UID root permettant un retour par changement d'UID. Une simple baisse de l'UID effectif, laissant un UID sauvegardé de 0, serait réversible. Le retour de setresuid() doit être vérifié avant de continuer. [7]

Ordre des opérations et moindre privilège

Pour un simple listage accessible à Alice, aucune opération métier n'exige root. Le privilège doit donc être abandonné dès le début, avant tout traitement complexe d'entrée et toute invocation externe. Après vérification des trois UID, un essai de réélévation doit échouer, puis seulement la commande sûre peut être invoquée. Si des groupes ou capacités supplémentaires ont été accordés, ils doivent aussi être traités ; les trois UID ne suffisent pas à décrire tous les privilèges d'un processus.

Dans le scénario observé, la transition et le message de vérification sont conformes au résultat attendu de T8. Cette conclusion porte sur le processus testé et ne constitue pas une garantie contre toute autre source de privilège du système.

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
[alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catal1 /tmp/test.txt ]
TEST ALICE
[alice@2eb8f1466ecb:~$ ~/exploit_e6a.sh ]
=== T4 : Injection de commande dans catal1 ===
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
[alice@2eb8f1466ecb:~$ ~/exploit_e6b.sh ]
=== T5 : Détournement par PATH sur vuln ===
UID effectif: 0
=== FAUX LS - execute par: uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam) ===
[alice@2eb8f1466ecb:~$ PATH=/tmp:$PATH /tmp/vuln_safe ]
UID effectif avant: 0
cap_leak ls mylib.so showuid.c test.txt vuln_final vuln_safe.c
cat myenv ping_cap showuids vuln vuln_final.c
catal1 mylib.c showuid showuids.c vuln.c vuln_safe
[alice@2eb8f1466ecb:~$ /tmp/vuln_final ]
AVANT abandon: ruid=1000 euid=0 suid=0
APRES abandon: ruid=1000 euid=1000 suid=1000
OK: réélévation impossible, privilège bien abandonné
cap_leak ls mylib.so showuid.c test.txt vuln_final vuln_safe.c
cat myenv ping_cap showuids vuln vuln_final.c
catal1 mylib.c showuid showuids.c vuln.c vuln_safe
alice@2eb8f1466ecb:~$

```

Figure 8 — Transition des trois UID vers 1000 et contrôle annoncé de non-réélévation.

5–6. Propriétés, corpus et bilan des essais

E9 — Propriétés explicites à vérifier

P1 : avant toute invocation externe, UID réel = UID effectif = UID sauvegardé = UID du lanceur non privilégié. P2 : toute commande externe utilise `execve()` avec un chemin absolu de confiance. P3 : l'environnement transmis est une liste blanche fixe. P4 : une entrée utilisateur ne peut pas introduire une commande supplémentaire. P5 : tout échec de l'abandon de privilège arrête le programme ; une tentative de retour à l'UID 0 doit échouer.

Corpus légué : huit cas documentés

Ce corpus de spécification est préparé à partir des scénarios du laboratoire. Les cas complémentaires sont proposés pour le rejeu dans l'environnement isolé ; ils ne sont pas présentés comme déjà exécutés. Les valeurs d'argument doivent être transmises littéralement, sans expansion par le shell du banc d'essai.

ID / entrée ou environnement	But et résultat attendu après correction
C1 : <code>/tmp/test.txt; id</code>	Injection par séparateur ; un seul nom de fichier, sans exécution de <code>id</code> . Entrée visible en E5/E6.
C2 : <code>/tmp/test.txt && id</code>	Injection conditionnelle ; aucun second programme. Cas à rejouer.
C3 : <code>/tmp/\${id}</code>	Substitution de commande ; aucun <code>id</code> exécuté. Cas à rejouer.
C4 : <code>/tmp/test.txt id</code>	Création d'un pipeline ; aucun pipeline. Cas à rejouer.
C5 : <code>PATH=/tmp:/usr/bin:/bin</code>	Faux <code>ls</code> dans <code>/tmp</code> ; sélection du chemin absolu de confiance. Scénario E6/E7.
C6 : <code>PATH=./usr/bin:/bin</code>	Faux exécutable dans le répertoire courant ; aucune substitution. Cas à rejouer.
C7 : <code>IFS=/</code>	Découpage influencé ; environnement fixe et absence de shell. Cas à rejouer.

Tableau des essais T1 à T8

Essai / preuve	Procédure et résultat observé / portée du verdict
T1 — E1, E2	Lire comptes, umask et modes : configuration présente ; création et suppression non visibles.
T2 — E3	getfacl et getcap : ACL ciblée et capacité présentes ; remplacement complet non visible.
T3 — E4	showuids : 1000 / 0 / 0 ; distinction des trois UID observée.
T4 — E6	exploit_e6a.sh : injection observée ; commande enfant sous Alice.
T5 — E6	exploit_e6b.sh : faux ls exécuté ; parent EUID 0, enfant sous Alice.
T6 — E5	LD_PRELOAD : aucun effet visible ; contrôle positif absent de la capture.
T7 — E7	PATH modifié : FAUX LS absent après correction ; rejeu de T4 non visible.
T8 — E8	vuln_final : 1000 / 1000 / 1000 ; non-réévaluation annoncée par le programme.

6. Limites de la correction

Une invocation sans shell empêche les métacaractères d'être interprétés comme commandes. Un chemin absolu supprime la recherche de l'exécutable dans PATH. L'abandon préalable des privilèges réduit l'impact d'une erreur ultérieure. Ces mesures ne remplacent pas une politique d'accès aux fichiers, la validation des options, la protection des exécutables de confiance ou la gestion des descripteurs déjà ouverts. Elles n'établissent pas non plus, à elles seules, l'absence de courses ou d'erreurs mémoire.

Les captures constituent des preuves de sorties observées. Les sources exactes de vuln, vuln_safe, vuln_final et les scripts exploit_e6a.sh et exploit_e6b.sh sont nécessaires pour vérifier complètement les mécanismes et reproduire la même série. Le corpus ci-dessus distingue donc les entrées visibles des essais complémentaires.

7. Travail d'équipe et démonstration

Le laboratoire a été réalisé en collaboration par Amel, Mat et Ibrahim. En raison de problèmes techniques, nous avons travaillé principalement sur un seul ordinateur, ce qui explique que l'historique Git ne reflète pas toujours les contributions individuelles. Chaque membre a participé à l'exécution des essais, à l'analyse des résultats et à la rédaction du rapport. Les décisions ont été prises collectivement et chaque membre est en mesure d'expliquer l'ensemble du travail remis.

Dépôt Git de l'équipe : <https://github.com/Ibrahim-44/Labceg4799>

8. Sources et déclaration d'usage de l'IA

Sources : énoncé du Laboratoire 1 CEG 4799 / CSI 4539 (Automne 2026); documentation et environnement SEED Labs; pages de manuel Linux utilisées pendant le laboratoire (chmod, umask, getfacl/setfacl, getcap/setcap, execve, getresuid/setresuid et ld.so).

Déclaration d'utilisation de l'IA : Nous avons utilisé ChatGPT comme outil d'appoint pour organiser la structure du rapport, reformuler certaines explications et améliorer la grammaire. Les commandes, essais, captures d'écran et résultats présentés proviennent de notre travail dans l'environnement du laboratoire. Nous avons vérifié le contenu final et nous sommes capables d'expliquer le code, les scripts et les concepts remis.

Annexe A — E2 : Permissions et umask

Reproduction agrandie de la capture présentée dans le corps du rapport.

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
-rw-r-----+ 1 alice alice  17 Sep 29 18:41 secret.txt
alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
alice@2eb8f1466ecb:~$ ls -la /home/alice/
total 44
drwxr-xr-x  3 alice alice 4096 Sep 29 18:41 .
drwxr-xr-x  1 root  root  4096 Sep 28 22:40 ..
-rw-----  1 alice alice 4267 Sep 29 18:05 .bash_history
-rw-r--r--  1 alice alice  220 Feb 25  2020 .bash_logout
-rw-r--r--  1 alice alice 3771 Feb 25  2020 .bashrc
-rw-r--r--  1 alice alice  807 Feb 25  2020 .profile
drwxr-xr-x  5 alice alice 4096 Sep 29 18:04 CEG4799-Lab1
-rwxrwxr-x  1 alice alice   96 Sep 29 15:56 exploit_e6a.sh
-rwxrwxr-x  1 alice alice  194 Sep 29 15:56 exploit_e6b.sh
-rw-r-----+ 1 alice alice  17 Sep 29 18:41 secret.txt
[alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
[alice@2eb8f1466ecb:~$ umask
0002
[alice@2eb8f1466ecb:~$ ls -la /shared
total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41 .
drwxr-xr-x 1 root  root    4096 Sep 29 18:58 ..
alice@2eb8f1466ecb:~$

```

Figure 2 — Permissions et umask

Figure A1 Trace E1

```
# group: alice

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...

alice@2eb8f1466ecb:~$ PATH=/tmp:$PATH /tmp/vuln_safe
UID effectif avant: 0
cap_leak ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv   ping_cap  showuids  vuln       vuln_final.c
catall   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$ /tmp/vuln_final
AVANT abandon: ruid=1000 euid=0 suid=0
APRES abandon: ruid=1000 euid=1000 suid=1000
OK: réélévation impossible, privilège bien abandonné
cap_leak ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv   ping_cap  showuids  vuln       vuln_final.c
catall   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$ cat /etc/passwd | grep -E "alice|bob|charlie"
alice:x:1000:1002::/home/alice:/bin/bash
bob:x:1001:1003::/home/bob:/bin/bash
charlie:x:1002:1004::/home/charlie:/bin/bash
alice@2eb8f1466ecb:~$ cat /etc/group | grep -E "devteam|security|alice|bob|charlie"
devteam:x:1000:alice,charlie
security:x:1001:bob,charlie
alice:x:1002:
bob:x:1003:
charlie:x:1004:
alice@2eb8f1466ecb:~$
```

Figure A2 - Trace E2

```
ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...

-rw-r-----+ 1 alice alice 17 Sep 29 18:41 secret.txt
alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
alice@2eb8f1466ecb:~$ ls -la /home/alice/
total 44
drwxr-xr-x 3 alice alice 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 28 22:40 ..
-rw----- 1 alice alice 4267 Sep 29 18:05 .bash_history
-rw-r--r-- 1 alice alice 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 alice alice 3771 Feb 25 2020 .bashrc
-rw-r--r-- 1 alice alice 807 Feb 25 2020 .profile
drwxr-xr-x 5 alice alice 4096 Sep 29 18:04 CEG4799-Lab1
-rwxrwxr-x 1 alice alice 96 Sep 29 15:56 exploit_e6a.sh
-rwxrwxr-x 1 alice alice 194 Sep 29 15:56 exploit_e6b.sh
-rw-r-----+ 1 alice alice 17 Sep 29 18:41 secret.txt
alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
alice@2eb8f1466ecb:~$ umask
0002
alice@2eb8f1466ecb:~$ ls -la /shared
total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 29 18:58 ..
alice@2eb8f1466ecb:~$
```

Figure A3 Trace E3

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...

-rwxrwxr-x 1 alice alice 194 Sep 29 15:56 exploit_e6b.sh
-rw-r-----+ 1 alice alice 17 Sep 29 18:41 secret.txt
[alice@2eb8f1466ecb:~$ umaskls -la /shared
-bash: umaskls: command not found
[alice@2eb8f1466ecb:~$ umask
0002
[alice@2eb8f1466ecb:~$ ls -la /shared
total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 29 18:58 ..
[alice@2eb8f1466ecb:~$ getfacl /home/alice/secret.txt
getfacl: Removing leading '/' from absolute path names
# file: home/alice/secret.txt
# owner: alice
# group: alice
user::rw-
user:bob:r--
group:---
mask::r--
other:---

[alice@2eb8f1466ecb:~$ getcap -r /tmp/ 2>/dev/null
/tmp/ping_cap = cap_net_raw+ep
alice@2eb8f1466ecb:~$

```

Figure A4 - Trace E4

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...

total 8
drwxrwxr-t 2 alice devteam 4096 Sep 29 18:41 .
drwxr-xr-x 1 root root 4096 Sep 29 18:58 ..
[alice@2eb8f1466ecb:~$ getfacl /home/alice/secret.txt
getfacl: Removing leading '/' from absolute path names
# file: home/alice/secret.txt
# owner: alice
# group: alice
user::rw-
user:bob:r--
group:---
mask::r--
other:---

[alice@2eb8f1466ecb:~$ getcap -r /tmp/ 2>/dev/null
/tmp/ping_cap = cap_net_raw+ep
alice@2eb8f1466ecb:~$ /tmp/showuid
Real UID: 1000
Effective UID: 0
[alice@2eb8f1466ecb:~$ /tmp/showuids
Real UID: 1000
Effective UID: 0
Saved UID: 0
alice@2eb8f1466ecb:~$

```

Figure A5 Trace E5

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...

[alice@2eb8f1466ecb:~$ getcap -r /tmp/ 2>/dev/null
/tmp/ping_cap = cap_net_raw+ep
alice@2eb8f1466ecb:~$ /tmp/showuid
Real UID: 1000
Effective UID: 0
alice@2eb8f1466ecb:~$ /tmp/showuids
Real UID:      1000
Effective UID: 0
Saved UID:     0
alice@2eb8f1466ecb:~$ # Surface 1 – injection via argument
alice@2eb8f1466ecb:~$ /tmp/catal1 "/tmp/test.txt; id"
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 2 – variable PATH
alice@2eb8f1466ecb:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 3 – liaison dynamique
alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catal1 /tmp/test.txt
TEST ALICE
alice@2eb8f1466ecb:~$ █

```

Figure A6 - Trace E6

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...

Saved UID:      0
alice@2eb8f1466ecb:~$ # Surface 1 – injection via argument
alice@2eb8f1466ecb:~$ /tmp/catal1 "/tmp/test.txt; id"
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 2 – variable PATH
alice@2eb8f1466ecb:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 3 – liaison dynamique
alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catal1 /tmp/test.txt
TEST ALICE
[alice@2eb8f1466ecb:~$ ~/exploit_e6a.sh
=== T4 : Injection de commande dans catal1 ===
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
[alice@2eb8f1466ecb:~$ ~/exploit_e6b.sh
=== T5 : Détournement par PATH sur vuln ===
UID effectif: 0
=== FAUX LS – execute par: uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam) ===
alice@2eb8f1466ecb:~$ █

```

Figure A7 Trace E7

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 2 — variable PATH
alice@2eb8f1466ecb:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@2eb8f1466ecb:~$
alice@2eb8f1466ecb:~$ # Surface 3 — liaison dynamique
alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catal1 /tmp/test.txt
TEST ALICE
alice@2eb8f1466ecb:~$ ~/exploit_e6a.sh
=== T4 : Injection de commande dans catal1 ===
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
alice@2eb8f1466ecb:~$ ~/exploit_e6b.sh
=== T5 : Détournement par PATH sur vuln ===
UID effectif: 0
=== FAUX LS — execute par: uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam) ===
alice@2eb8f1466ecb:~$ PATH=/tmp:$PATH /tmp/vuln_safe
UID effectif avant: 0
cap_leak  ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv    ping_cap  showuids   vuln      vuln_final.c
catal1   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$

```

Figure A8 - Trace E8

```

ibrahimb — alice@2eb8f1466ecb: ~ — docker start -ai ceg4799-lab1 — 8...
alice@2eb8f1466ecb:~$ LD_PRELOAD=/tmp/mylib.so /tmp/catal1 /tmp/test.txt
TEST ALICE
alice@2eb8f1466ecb:~$ ~/exploit_e6a.sh
=== T4 : Injection de commande dans catal1 ===
TEST ALICE
uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam)
alice@2eb8f1466ecb:~$ ~/exploit_e6b.sh
=== T5 : Détournement par PATH sur vuln ===
UID effectif: 0
=== FAUX LS — execute par: uid=1000(alice) gid=1002(alice) groups=1002(alice),1000(devteam) ===
alice@2eb8f1466ecb:~$ PATH=/tmp:$PATH /tmp/vuln_safe
UID effectif avant: 0
cap_leak  ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv    ping_cap  showuids   vuln      vuln_final.c
catal1   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$ /tmp/vuln_final
AVANT abandon: ruid=1000 euid=0 suid=0
APRES abandon: ruid=1000 euid=1000 suid=1000
OK: réélévation impossible, privilège bien abandonné
cap_leak  ls      mylib.so  showuid.c  test.txt  vuln_final  vuln_safe.c
cat      myenv    ping_cap  showuids   vuln      vuln_final.c
catal1   mylib.c  showuid   showuids.c vuln.c     vuln_safe
alice@2eb8f1466ecb:~$

```